

each request into a SQL query made to XTRA1 to get data from the database. That way, only the code of the `Database` object knew how to access the XTRA using its API.

The program worked well, and the customers enjoyed using the product. But every once in a while, we would run into errors in the data we got back from the database. I contacted the XTRA1 developer and gave many easily reproducible examples of the problems. Unfortunately, the developer never addressed these issues.

Because of the lack of response, we eventually decided to purchase a different database XTRA, XTRA2, to serve this purpose. XTRA2 worked in a similar way but had some subtle differences in how it was initialized and required some minor changes in the way that SQL queries were built.

Because the `Database` class encapsulated all the details of communicating with the XTRA, we were able to make all the necessary changes to work with XTRA2 only in the `Database` class. We did not change a single line in the main program (the client code).

In this case, I was both the `Database` class developer and the client software developer. If my client code had used the names of instance variables in the class, I would have had to crawl through the program, modifying each line of relevant code. Using encapsulation with a class saved me from countless hours of reworking and testing.

As a follow-up to the story, while XTRA2 worked well, that company eventually went out of business and I had to go through the same process again. Once again, because of encapsulation, only the code of the `Database` class was modified to work with XTRA3.

Abstraction

Abstraction is another OOP concept closely related to encapsulation; many developers consider it to be the fourth tenet of OOP.

Whereas encapsulation is about implementation, hiding the details of the code and data that make up a class, *abstraction* is about the client's view of a class. It's about the perception of a class from the outside.